TECNOLÓGICO NACIONAL DE MÉXICO

TAREA 2.2: TRANSICIÓN DE DIAGRAMA ENTIDAD-RELACIÓN A NOTACIÓN UML

Asignatura: Sistemas de Bases de Datos

Docente: Ruben Miguel Riojas Rodríguez

Alumno: Eduardo Salazar Martínez

1. Introducción y Contextualización del Caso

El diseño de sistemas de información modernos exige salvar la brecha entre la persistencia de datos relacional y la arquitectura de software en capa de aplicación. En la actividad previa (Tarea 2.1), se desarrolló el modelo conceptual Entidad-Relación Extendido (EER) para la **Clínica Salud Total**, resolviendo la ineficiencia de registros fragmentados mediante la consolidación de expedientes, personal médico, citas, diagnósticos y prescripción de medicamentos.

Se establece la transición formal desde el modelo E-R/EER hacia el **Diagrama de Clases en Notación UML (Unified Modeling Language)**. Esta transformación traslada las estructuras de datos estáticas (entidades, claves, relaciones) hacia un paradigma Orientado a Objetos (POO), incorporando visibilidad de atributos, métodos de dominio, comportamiento dinámico y tipos de asociación explícitos (herencia, agregación y composición).

> **■ OBJETIVO DE LA TRANSICIÓN**
> La transición de E-R a UML traslada un esquema puramente relacional enfocado en tablas y restricciones SQL hacia una arquitectura de clases con encapsulamiento, abstracción y métodos de negocio que facilitan la implementación mediante patrones de Mapeo Objeto-Relacional (ORM).

2. Correspondencia Directa entre el Modelo E-R/EER y Notación UML

Para garantizar una transformación rigurosa sin pérdida de semántica, se define una matriz de equivalencias entre los elementos conceptuales del modelo Entidad-Relación Extendido y sus contrapartes en el Diagrama de Clases UML:

Elemento Modelo E-R / EER	Notación Diagrama de Clases UML	Descripción y Mapeo en Clínica Salud Total
Entidades Principales (PERSONA, CITA, etc.)	Clases de Dominio (class Persona, class Cita)	Transformación de entidades relacionales en clases estructuradas con atributos privados y métodos públicos.
Superclase / Subclases Jerarquía (d, t)	Herencia / Generalización (Persona <|-- Paciente)	Mapeo directo de la especialización disjunta y total. Paciente y Médico heredan atributos y operaciones de Persona.
Atributos Simples y Claves (PK, FK, VARCHAR, INT)	Propiedades y Visibilidad (- idPersona: int [PK])	Los atributos de entidad pasan a ser propiedades privadas (-) con tipo de dato explícito e identificador de rol.
Atributos Derivados (Edad en Persona)	Métodos / Operaciones (+ calcularEdad(): int)	Los atributos calculados dinámicamente dejan de ser campos almacenados y se convierten en operaciones de clase.
Atributos Multivalorados (Teléfono, Especialidad)	Composición / Colecciones (Persona 1 *-- 1..* PersonaTelefono)	Se descomponen mediante relaciones de composición o listas administradas de objetos dependientes.
Agregación EER (CONSULTA origina de CITA)	Agregación UML (o--) (Cita 1 o-- 0..1 Consulta)	La Consulta Médica agrega la interacción entre Cita, Médico y Paciente en una relación todo-parte de acoplamiento débil.
Relación 1:N Fuerte (CONSULTA -> RECETA)	Composición UML (*--) (Consulta 1 *-- 0..* Receta)	La Receta no posee existencia independiente fuera del contexto de una Consulta Médica activa (vida dependiente).
Relación M:N con Atributos (RECETA <-> MEDICAMENTO)	Clase de Asociación (DetalleRecetaMedicamento)	La relación M:N se resuelve mediante una clase asociativa que encapsula dosis, frecuencia y duración del tratamiento.

3. Especificación Detallada de Clases y Estructura UML

A continuación se presenta la definición formal de las clases del sistema de la Clínica Salud Total en notación UML, detallando visibilidad, tipos de datos, multiplicidades y firmas de métodos:

3.1. Clases de Dominio y Jerarquía de Herederos

Clase: Persona (Clase Abstracta Base)

Atributos Privados (-)	Métodos Públicos (+)
- idPersona: int {PK} - primerNombre: String - apellidoPaterno: String - apellidoMaterno: String [0..1] - calle: String - numero: String - colonia: String - codigoPostal: String - fechaNacimiento: Date	+ calcularEdad(): int + obtenerNombreCompleto(): String + obtenerDireccionCompleta(): String

Clase: Paciente (Subclase heredada de Persona)

Atributos Privados (-)	Métodos Públicos (+)
- numExpediente: String {PK, Format: 'EXP-YYYY-XXXX'} - tipoSangre: String {CHECK in ('A+','A-','O+','O-',...)}	+ obtenerHistorialCitas(): List<Cita> + validarExpediente(): boolean

Clase: Medico (Subclase heredada de Persona)

Atributos Privados (-)	Métodos Públicos (+)
- numCedula: String {PK} - fechaContratacion: Date	+ obtenerEspecialidades(): List<String> + asignarDepartamento(depto: Departamento): void

Clase: Departamento

Atributos Privados (-)	Métodos Públicos (+)
- idDepto: int {PK} - nombreDepto: String {UNIQUE} - ubicacion: String	+ listarMedicos(): List<Medico> + obtenerEstadisticasAtencion(): Map

Clase: Cita

Atributos Privados (-)	Métodos Públicos (+)
- folioCita: int {PK} - fecha: Date - hora: Time - estado: String {DEFAULT 'Programada'}	+ reprogramar(nuevaFecha: Date, nuevaHora: Time): void + cancelar(): void + registrarAtencion(): Consulta

Clase: Consulta (Clase de Agregación)

Atributos Privados (-)	Métodos Públicos (+)
- idConsulta: int {PK} - diagnostico: String - costoBase: double {costoBase >= 0}	+ generarReceta(): Receta + calcularCostoTotal(): double

Clase: Receta

Atributos Privados (-)	Métodos Públicos (+)
- folioReceta: int {PK} - fechaEmision: Date	+ agregarMedicamento(med: Medicamento, dosis: String, freq: String, duracion: int): void + obtenerDetalles(): List<DetalleRecetaMedicamento>

Clase: Medicamento

Atributos Privados (-)	Métodos Públicos (+)
- codigoMed: int {PK} - nombreComercial: String - formula: String	+ consultarStock(): int + obtenerPresentacion(): String

Clase: DetalleRecetaMedicamento (Clase Asociativa M:N)

Atributos Privados (-)	Métodos Públicos (+)
- dosis: String - frecuencia: String - duracionDias: int	+ obtenerResumenTratamiento(): String

4. Matriz Comparativa Detallada: Modelo E-R vs. Diagrama de Clases UML

Para profundizar en las diferencias conceptuales, metodológicas y operativas entre ambos enfoques, se estructura la siguiente matriz de análisis comparativo:

Dimensión / Criterio	Modelo Entidad-Relación (E-R / EER)	Diagrama de Clases UML	Análisis de Impactización
Propósito Principal	Diseñar el esquema conceptual de persistencia física en un Sistema Gestor de Bases de Datos Relacional (SGBD).	Modelar la arquitectura estática y dinámica de las clases del dominio del software orientado a objetos.	E-R busca la estructuración del almacenamiento; UML busca la estructura del código ejecutable.
Enfoque de Datos y Comportamiento	Estático y pasivo: se enfoca únicamente en entidades, atributos, llaves y restricciones de integridad.	Dinámico e activo: integra estado (propiedades) y comportamiento ejecutable (métodos y operaciones).	UML añade lógica de negocio dentro de los mismos bloques mediante encapsulamiento POO.
Encapsulamiento y Visibilidad	No existe encapsulamiento: todos los campos de una tabla son públicos y accesibles por SQL.	Encapsulamiento estricto: define atributos privados (-), protegidos (#) y métodos públicos (+).	UML garantiza la integridad de datos impidiendo modificaciones directas a propiedades.
Expresividad de Relaciones Avanzadas	Soporta Especialización (d, t) y Agregación básica mediante tablas intermedias y FKs.	Soporta Herencia (<|--), Composición (*--), Agregación (o--), Interfaces y Asociaciones complejas.	UML diferencia semánticamente entre la dependencia vital (Composición) y el agrupamiento débil (Agregación).
Manejo de Derivados y Calculados	Atributos derivados (Edad) no se almacenan física ni formalmente en tablas 3FN.	Se representan explícitamente mediante métodos de cálculo de clase (+calcularEdad(): int).	En UML los atributos calculados son responsabilidad de la lógica de la capa de aplicación.

5. Análisis de Coincidencias para el Desarrollo Orientado a Objetos (POO & ORM)

5.1. Resolución del Desfasaje Impedante (Impedance Mismatch)

El denominado **desfasaje impedante relacional-objeto** representa la incompatibilidad conceptual entre el modelo relacional (basado en tablas, tuplas, álgebras relacionales y claves primarias/foráneas) y el paradigma orientado a objetos (basado en objetos, identidad en memoria, grafos de referencias y polimorfismo). La conversión a notación UML permite anticipar y resolver estos conflictos antes de escribir código de persistencia.

5.2. Estrategia de Mapeo de Herencia en ORM

La especialización disjunta y total **(d, t) de la superclase PERSONA** hacia las subclases PACIENTE y MÉDICO puede implementarse en frameworks ORM (tales como Hibernate, Entity Framework o SQLAlchemy) utilizando tres patrones estándar:

- **Joined Table (Tabla por Subclase):** Se crea una tabla para la superclase PERSONA y tablas independientes para PACIENTE y MÉDICO vinculadas por la misma llave primaria (ID_Persona). Es la estrategia más limpia, evita nulos sobrantes y preserva la normalización 3FN alcanzada en el modelo relacional.
- **Single Table (Tabla Única por Jerarquía):** Consolida todas las propiedades en una sola tabla con una columna discriminadora ('TIPO_PERSONA'). Es eficiente en consultas pero introduce múltiples valores nulos en columnas específicas.
- **Table Per Concrete Class:** Elimina la tabla superclase y duplica los atributos comunes en las tablas de las subclases concretas. Dificulta las consultas polimórficas sobre todas las personas.

5.3. Trazabilidad de Reglas de Negocio en la Capa de Dominio

La notación UML permite trasladar las Reglas de Negocio Formales (RN-01 a RN-08) directamente a constructos POO:

✓ **RN-01 (Jerarquía Disjunta):** Se implementa mediante una clase abstracta Persona con un constructor de ámbito protegido (protected), impidiendo la instanciación directa de personas genéricas.

✓ **RN-02 (Expedientes de Pacientes):** La propiedad numExpediente se valida mediante una expresión regular en el setter de la clase Paciente, asegurando la sintaxis 'EXP-YYYY-XXXX'.

✓ **RN-04 (Estructura Departamental):** La relación de agregación Departamento o-- Medico se gestiona mediante métodos encadenados (+asignarDepartamento()) que mantienen la consistencia bidireccional.

✓ **RN-06 y RN-07 (Consulta y Recetas):** La composición Consulta *-- Receta garantiza que las recetas se instancien mediante el método de fábrica +generarReceta() dentro del contexto de una consulta finalizada.

6. Conclusiones Técnicas

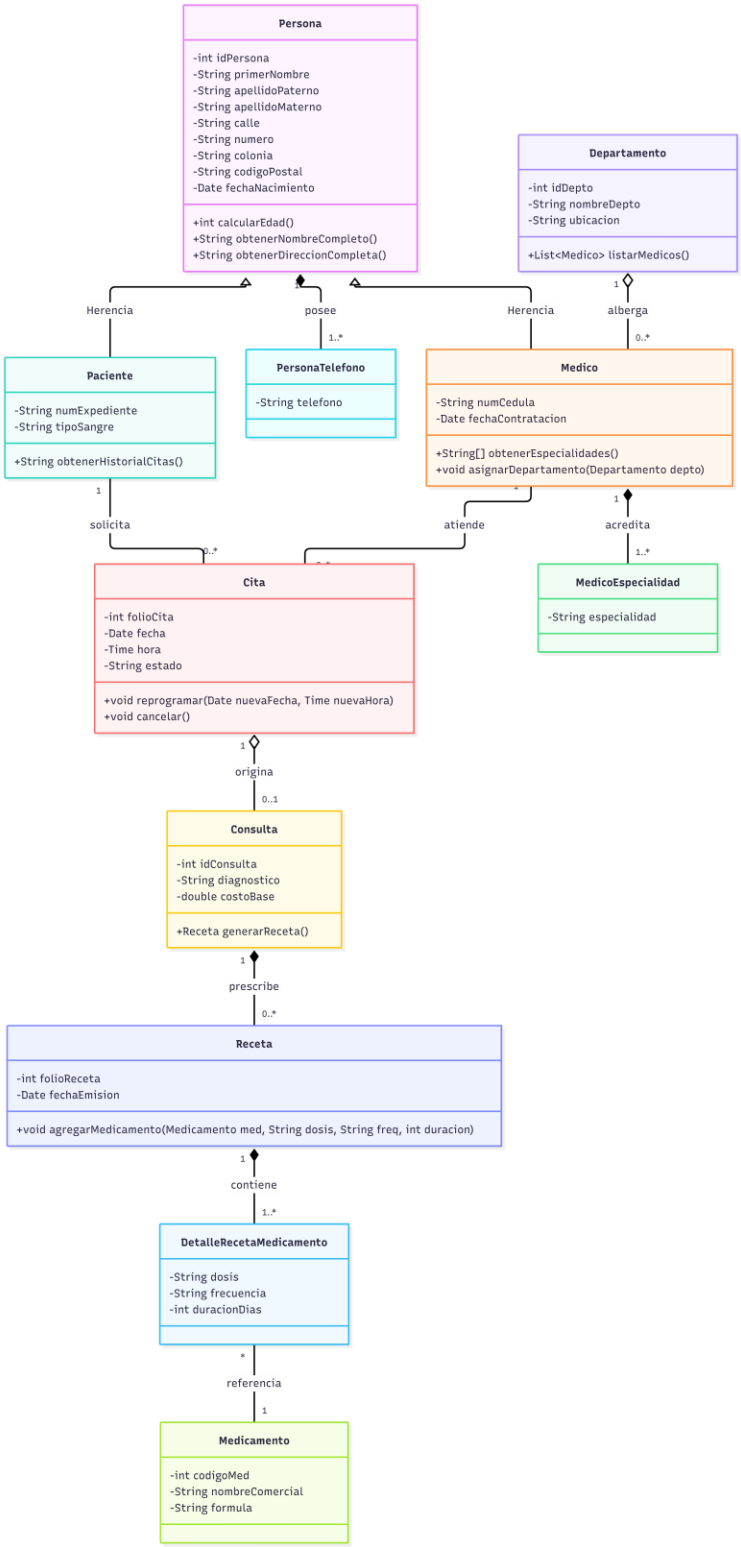
La transición del Diagrama Entidad-Relación Extendido hacia la Notación UML demuestra que ambos modelos no son excluyentes, sino complementarios en el ciclo de vida del desarrollo de software médica:

1. El modelo E-R/EER constituye el cimiento óptimo para garantizar la integridad referencial, la normalización (1FN/3FN) y la eficiencia del almacenamiento físico en el SGBD.

2. El Diagrama de Clases UML proporciona el plano de arquitectura necesario para implementar la capa de lógica de negocio en la aplicación, encapsulando comportamientos y habilitando el uso de patrones de diseño (Domain-Driven Design).

3. La correspondencia formal establecida facilita el uso de herramientas de Mapeo Objeto-Relacional (ORM), reduciendo significativamente el tiempo de desarrollo y los errores en tiempo de ejecución.

Figura 1. Diagrama UML del Sistema Clínica Salud Total.



7. Referencias Bibliográficas (APA 7.ª Edición)

Coronel, C., & Morris, S. (2019). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.

Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Pearson Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2017). Fundamentos de sistemas de bases de datos (7.ª ed.). Pearson Educación.

Object Management Group. (2017). Unified Modeling Language (UML) Specification (Version 2.5.1). OMG.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). Fundamentos de bases de datos (6.ª ed.). McGraw-Hill Education.